

TAG: Applying DreamerV3 to a Labyrinth with a Top-Mounted Rotary Inverted Pendulum

Trung Bao Truong, Thanh Dat Nguyen

Control, Optimization, and Online Learning (COOL) for Autonomy Lab

Department of Aerospace Engineering and Engineering Mechanics

The University of Texas at Austin

Abstract

TAG is a real labyrinth robot designed for reinforcement-learning experiments. Two servos tilt the maze, a camera tracks the marble, and a Rotary Inverted Pendulum is mounted at the center of the board. The pendulum makes the task harder because its motion shakes the same structure that moves the marble. We use DreamerV3 as the high-level controller. It receives the maze image, board angles, marble position and measured velocity, future path points, and optional pendulum data. Its action is converted into desired pitch and roll angles for the two maze servos. This paper explains how the learning algorithm is connected to the real hardware, which neural-network blocks are used inside DreamerV3, why we currently use its recurrent world model instead of a Transformer, and what we learned from the first training run. We have also built a MuJoCo version of the maze. Its purpose is not to reproduce every detail of the real servo mechanism. Instead, it provides a safe place to learn how the marble moves on a tilted board, interacts with walls and holes, and performs reusable skills such as braking and turning. Real hardware replay is still needed to learn the actual servo-to-board response and other mechanical effects before deployment.

1. Introduction

The goal of TAG is easy to describe: move a marble from the start of a maze to the end without falling into a hole. The real problem is much harder. The marble has momentum, the board has actuator delay, the camera is not perfect, and contact with a wall can change the motion quickly. In our platform, the Rotary Inverted Pendulum (RIP) is mounted directly on the maze. Its motor and rotating mass add vibration and reaction torque to the board.

CyberRunner showed that model-based reinforcement learning can learn a real labyrinth directly from camera experience [2, 3]. TAG follows the same general idea, but our platform adds the top-mounted pendulum, a different servo system, a compact learned marble detector, stronger state checks, remote GPU training, and safety tools for long experiments. The goal is not to claim that TAG already performs better. The goal is to build a platform where we can study learning under disturbance and later test transfer to new maps.

2. TAG Platform

At each control step, the camera sends an image to the robot computer. The state estimator finds the marble, estimates the board pose, and converts the result into maze coordinates. The observation is sent to the GPU server, where DreamerV3 chooses the next board-angle command. The two Hiwonder LX-16A servos then move to the requested pitch and roll positions. The new observation, reward, and action are stored in replay so the same physical experience can be used for many training updates.

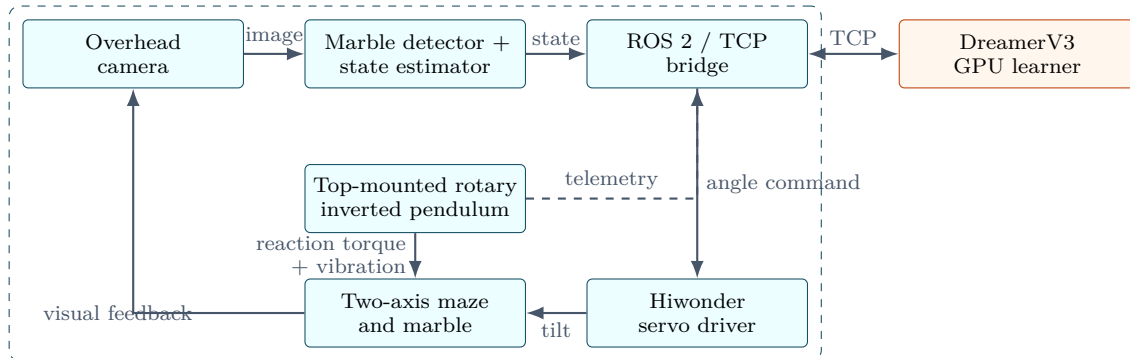


Figure 1: TAG data and control flow. DreamerV3 selects desired maze angles, while the low-level servo loops execute those positions.

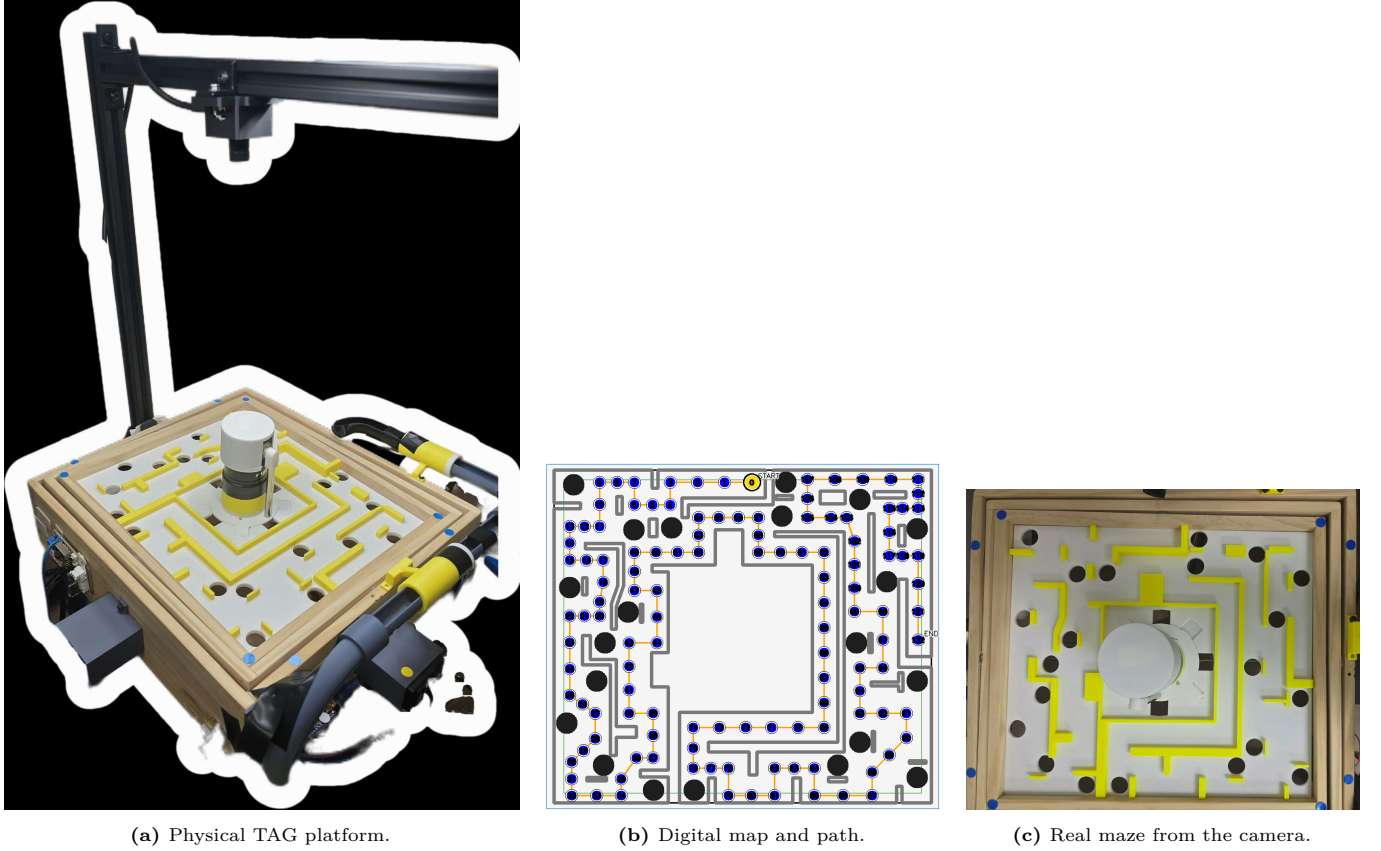


Figure 2: Hardware and map calibration. The software geometry must match the physical walls, holes, and path closely enough for reward and safety checks to be correct.

2.1 How TAG differs from CyberRunner

The main differences are platform changes, not a claim of better final performance.

Table 1: Main extensions from the CyberRunner foundation to TAG.

Area	CyberRunner foundation	TAG extension
Mechanical system	Two-axis physical labyrinth	Adds a rotary inverted pendulum on the maze, creating coupled vibration and reaction torque
Actuation	Dynamixel-based two-axis interface	Uses Hiwonder LX-16A servos with calibrated pitch and roll position commands
Perception	Camera-based maze state	Adds an ONNX marble detector, marker-based homography, short-occlusion handling, and rejection of impossible position jumps
Operation	Real-time model-based RL	Separates robot and GPU server through TCP and adds motor cooldown, timeout, stuck detection, and checkpoint management

3. How We Use DreamerV3

We use DreamerV3 only for the high-level maze decision. It does not replace the servo position controllers, and it does not have to replace the controller that stabilizes the pendulum. Its job is to choose the desired board pitch and roll for the next control step.

3.1 Observation

The current observation combines an image with measured numbers:

$$o_t = \{I_t, \alpha_t, \beta_t, x_t, y_t, v_{x,t}, v_{y,t}, g_t, p_t, d_t\}. \quad (1)$$

Here, I_t is a small local maze image, α_t and β_t are board angles, (x_t, y_t) is marble position, and $(v_{x,t}, v_{y,t})$ is the measured marble velocity from the state estimator. The vector g_t contains several future path points relative to the marble, p_t is path progress, and d_t is optional pendulum or vibration data.

Adding measured velocity is useful because position alone does not tell the controller where the marble is moving. Two frames can show the marble at almost the same point, but one marble may be moving toward a hole while the other is slowing down. Giving velocity directly should make braking and cornering easier to learn. Because velocity estimates can be noisy, we will compare training with and without (v_x, v_y) rather than assuming it always helps.

3.2 Action and real-time step

DreamerV3 outputs two normalized values:

$$a_t = [a_t^{(\alpha)}, a_t^{(\beta)}] \in [-1, 1]^2. \quad (2)$$

These values are mapped to safe pitch and roll setpoints. This is position control, not velocity control. We use position commands because the requested board angle is clear and repeatable. A rate limit may smooth sudden changes, but the policy still chooses the target angle.

One environment step is simple:

1. read the camera and measured state;
2. send the observation to DreamerV3;
3. convert the two actions into safe angle setpoints;
4. hold the command for one control interval;
5. measure the new marble state;
6. calculate reward and check for success, a hole, timeout, or invalid sensing;
7. save the transition in replay.

3.3 What neural networks are inside DreamerV3?

DreamerV3 is not one large neural network. It is a group of smaller networks that work together. For TAG, the important parts are shown in Figure 3.

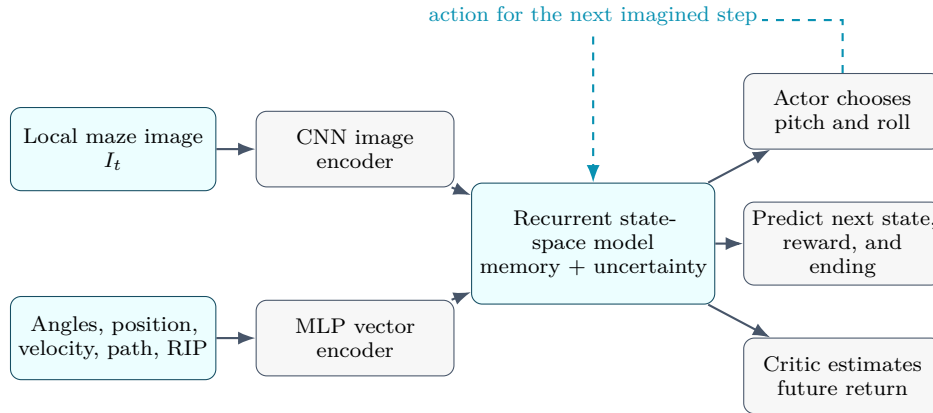


Figure 3: The main neural-network blocks used by DreamerV3 in TAG. The recurrent world model remembers recent motion. During imagination, the actor selects an action, the world model predicts what may happen next, and the critic estimates the future return.

Image encoder. A Convolutional Neural Network (CNN) reads the small maze image. Convolution filters are useful here because the same wall, corner, or hole pattern can appear at different image locations. The CNN compresses the image into a shorter feature vector instead of passing every pixel directly to the controller.

Vector encoder. A Multi-Layer Perceptron (MLP) reads the measured values: board angle, marble position, marble velocity, path points, progress, previous action, and optional pendulum data. An MLP is suitable because these inputs already have clear numerical meanings.

Recurrent world model. The core memory is a Recurrent State-Space Model (RSSM). “Recurrent” means that it carries a hidden memory from one control step to the next. This memory helps represent information that is not fully visible in a single camera frame, such as marble momentum, servo delay, a recent wall impact, or the phase of the pendulum disturbance. The RSSM combines a deterministic recurrent state with a stochastic latent state. The deterministic part stores a running summary of the past. The stochastic part allows the model to represent uncertainty when several futures are possible [1].

Prediction heads. Small MLP heads predict the next latent state, reward, and whether the episode is likely to continue. During training, the model also learns to reconstruct or explain the observation. These predictions create the “world model” used for imagined rollouts.

Actor and critic. The actor is an MLP that outputs a probability distribution for the two continuous pitch and roll actions. The critic is another MLP that estimates the future reward expected from the current latent state. DreamerV3 trains these two networks on trajectories generated inside the learned world model, so one real transition can support more than one policy update.

3.4 Why we do not use a Transformer now

A Transformer could also be used as the temporal model, and it may become useful when TAG is trained on many maps and much larger datasets. We do not use one in the current system for four practical reasons.

First, the recurrent RSSM is already part of DreamerV3 and has been tested for continuous-control tasks. It updates one compact hidden state at every step, which fits an online robot loop. Second, TAG already provides marble velocity and short path context, so the controller does not need to search through a very long history for basic motion information. Third, a Transformer attends to a sequence of earlier tokens. This can improve long-range memory, but its computation and memory grow with the context length, while recurrent models are efficient during step-by-step imagination [5]. Fourth, our real dataset is still small compared with the large and diverse datasets where Transformers are usually most useful.

This does not mean that a Transformer is always worse. Research has shown that Transformer world models can help some long-horizon problems, but the architecture must be designed carefully; a standard Transformer does not automatically produce better policy gradients [6]. For an undergraduate project, the recurrent DreamerV3 model gives us a simpler and better-tested starting point. A fair future study could keep the same TAG observations and replay data, then compare RSSM, Transformer, and possibly a structured state-space model using prediction error, success rate, inference time, and GPU memory.

3.5 Reward and training

The main reward is progress along a numbered path. A one-time bonus is given when the marble reaches a new checkpoint. Falling into a hole ends the episode with a penalty. Timeout and stuck checks prevent the robot from running forever without useful movement. Anti-shortcut checks reject large path jumps that are caused by a detector error or by crossing through an invalid part of the map.

We first train with the pendulum motor disabled. This lets the policy learn the maze, gravity, friction, camera geometry, and servo delay. After the base policy works, we can fine-tune with the pendulum active. Compatible old replay should be kept because it still contains useful ball and board dynamics. New pendulum-active replay then teaches the extra disturbance.

3.6 Why DreamerV3?

No single controller is best for every part of TAG. We still use normal position control inside the servos. PID or LQR is useful for a known local target, but it does not decide how to move through a full maze with holes and impacts. Model Predictive Control is a strong alternative, but it needs a good model of rolling contact, friction, delay, and pendulum disturbance. PPO is easier to understand, but it does not reuse old physical data as strongly. SAC is an important model-free baseline because it is off-policy, but it still learns mainly from real replay transitions. DreamerV3 is attractive because it reuses replay and trains the policy using predicted future trajectories [1]. Its recurrent state is also a natural fit for hidden momentum, delay, and pendulum motion. The tradeoff is more software complexity and the risk of learning an inaccurate world model.

Practical reason for our choice. Physical data are expensive. Motors heat up, episodes may require resets, and unsafe exploration can drop the marble into a hole. DreamerV3 gives us a way to learn more from each real transition, while standard controllers remain responsible for the simple low-level loops.

4. Preliminary Training Result

Figure 4 shows one long TensorBoard run. The success rate stayed close to zero for most of training and then increased after about three million environment steps. The smoothed success-rate value shown at step 4,240,020 is 0.3792. Episode duration changed throughout training, and the smoothed log reward increased before dropping near the end. These plots show that the policy learned useful behavior, but they are not enough to make a final performance claim. The raw curves have large variation, and only one run is shown.

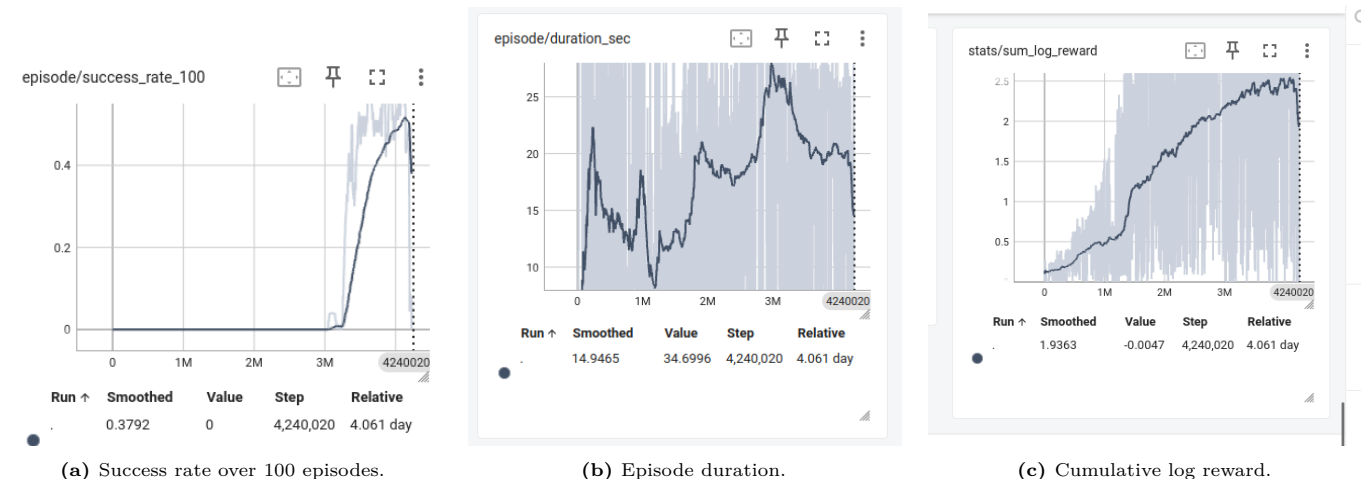


Figure 4: Preliminary TensorBoard curves from one baseline run at 4,240,020 environment steps. Dark lines are smoothed values. Final evaluation should use repeated runs and fixed starting positions.

The next evaluation should report success rate, completion time, maximum progress, hole rate, and physical training steps. We also plan a small ablation study: the same configuration with and without measured marble velocity. This will show whether (v_x, v_y) actually improves learning speed and stability.

5. Future Work: MuJoCo, Physics-Guided Learning, and New Maps

The current DreamerV3 agent has to learn most of the marble dynamics from replay. This works, but collecting millions of steps on the real robot takes time and heats the motors. Our future plan is to combine three sources of knowledge: a MuJoCo simulation, a simple ball-on-plate model, and real hardware data. The simulator will give the policy a safe place to practice, while real measurements will keep the model connected to the actual TAG system.

5.1 MuJoCo simulation for marble-board skill learning

We have built a MuJoCo version of TAG with a two-axis board, a rolling ball, wall collisions, physical holes, and position-controlled tilt joints [7, 8]. The simulator uses the same type of maze geometry, route targets, marble state, and desired pitch-roll action as the real environment. This makes it useful for training on many maze layouts without risking the hardware.

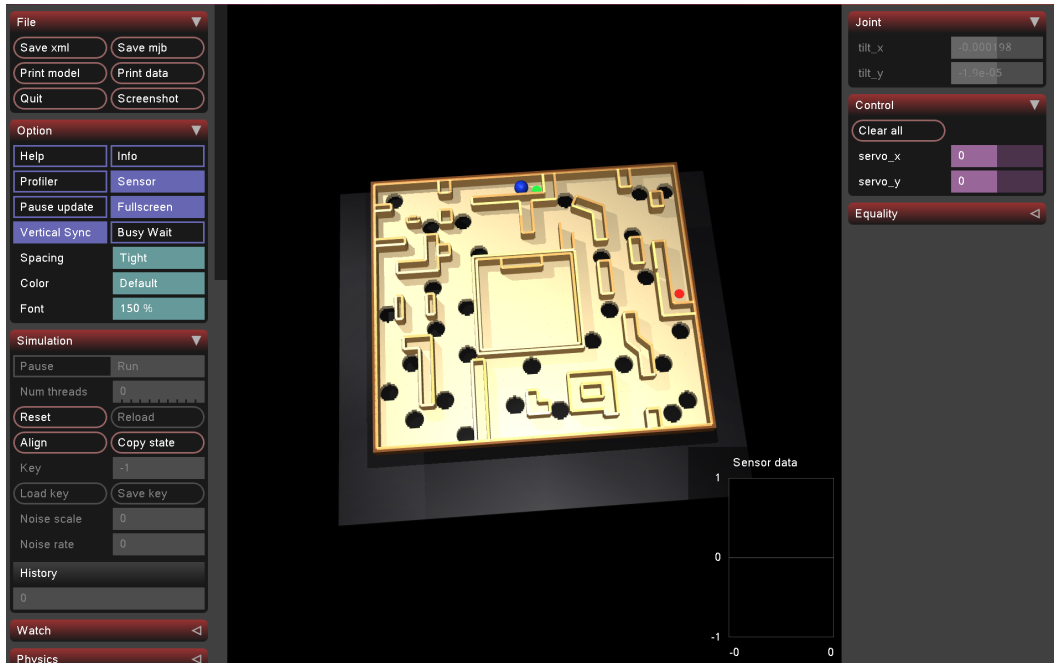


Figure 5: Current MuJoCo maze prototype. The simulation is mainly used to learn marble motion on a tilted board, wall interaction, braking, turning, and route-following skills. It is not expected to reproduce the real servo mechanism exactly.

The important limitation is the connection between the servo command and the real board motion. The physical system contains the servo controller, linkage geometry, backlash, frame compliance, load-dependent delay, and vibration. Reproducing all of these effects exactly in MuJoCo would require a much more detailed mechanical model and may still be inaccurate. Therefore, we do not treat the simulator as a complete copy of TAG.

Instead, we separate the problem into two parts. MuJoCo learns the more general *marble-board behavior*: how measured pitch and roll accelerate the marble, how momentum changes near a corner, and how the marble interacts with walls and holes. The real robot then teaches the policy the hardware-specific behavior: how a desired angle command becomes the measured board angle, how much delay or overshoot occurs, and how the pendulum and frame affect the motion.

This separation is possible because TAG measures the board angles. In simulation, the policy can observe simulated pitch and roll. On the real robot, it observes the measured pitch and roll rather than assuming that the board instantly reaches the commanded angles. The action remains a desired position command, but the recurrent world model can use real replay to learn the delay between command and motion.

The planned process is:

1. collect measured board angles, marble position, and marble velocity from safe hardware runs;
2. tune the MuJoCo ball, friction, gravity, wall-contact, and hole behavior using the *measured* board motion as input;
3. randomize a simple range of board response, delay, and observation noise instead of trying to copy the exact servo mechanism;
4. pretrain DreamerV3 on generated mazes and short marble-control skills;
5. transfer the checkpoint to TAG and fine-tune with real replay so the world model learns the true servo-to-board response.

The expected benefit is not perfect zero-shot transfer. The goal is to start real training with useful marble-control behavior, so fewer physical steps are needed to learn a new map.

5.2 What real data are used for the simulation

Real measurements are still important, but different data have different purposes. Marble trajectories are used to improve the ball-on-plate and contact model. Servo measurements are used mainly to define realistic ranges for delay and board response, not to claim that the complete servo mechanism has been reproduced.

Table 2: How real TAG measurements support the MuJoCo model and transfer.

Real measurement	Use in simulation or training	Simple collection test
Measured board angle with marble position and velocity	Fit tilt-to-acceleration scale, rolling resistance, and ball response	Release the marble under several small measured pitch and roll angles
Low-speed wall impacts	Tune contact damping and energy loss	Guide the marble toward one wall at controlled speeds
Commanded versus measured board angle	Estimate a range of lag, rate limit, and overshoot for randomization; the exact mechanism is learned from real replay	Send small step and slow sine position commands
Camera state and timestamps	Add observation delay, noise, and occasional dropout	Compare estimator output with synchronized video and commands
Pendulum encoder or accelerometer data	Define disturbance ranges for later robust training	Repeat matched runs with the pendulum disabled and active

The result is a useful but intentionally simplified simulator. It should reproduce the main marble response to board orientation and local geometry. Hardware-specific servo behavior remains part of the real-world adaptation problem.

5.3 A simple ball-on-plate model

A useful physical state is

$$s_t = [x_t, y_t, v_{x,t}, v_{y,t}, \alpha_t, \beta_t], \quad (3)$$

where (x, y) and (v_x, v_y) are the measured marble position and velocity, and (α, β) are the measured board angles. The action remains the commanded pitch and roll position. For small board angles, a simple model can use

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad (4)$$

$$\dot{v}_x = k_x g \sin(\alpha) - c_x v_x, \quad \dot{v}_y = k_y g \sin(\beta) - c_y v_y. \quad (5)$$

The constants k_x and k_y describe how board tilt changes acceleration, while c_x and c_y roughly describe rolling resistance. They can be fitted from short real experiments with the pendulum motor disabled. This model does not need to be perfect. It only needs to predict the main direction and size of the marble acceleration.

5.4 Physics prediction plus a learned correction

The simple model will be wrong near walls, during impacts, and when the pendulum shakes the frame. Instead of asking the equations to explain everything, we can train a small residual network to predict the error:

$$\hat{s}_{t+1} = f_{\text{physics}}(s_t, a_t) + f_{\text{residual}}(s_t, a_t, d_t, m_t). \quad (6)$$

Here, d_t contains pendulum or vibration measurements and m_t describes the local maze geometry. The physics term gives a first prediction. The residual term learns friction changes, servo delay, frame flexibility, wall contact, camera error, and reaction torque from the rotary pendulum.

This design is easier to train than a completely separate high-fidelity simulator. We do not need to model every contact exactly. We only use the equations for the behavior that is already understood, and use data for the remaining difference.

5.5 How we would connect the model to DreamerV3

The first implementation does not require changing the internal RSSM. At every real step, the environment calculates the physics-predicted next position and velocity and appends them to the vector observation:

$$o'_t = \{o_t, \hat{x}_{t+1}^{\text{phys}}, \hat{y}_{t+1}^{\text{phys}}, \hat{v}_{x,t+1}^{\text{phys}}, \hat{v}_{y,t+1}^{\text{phys}}\}. \quad (7)$$

DreamerV3 can use this prediction as an extra hint, while its recurrent world model is still free to disagree when the real data show different behavior.

A later version can add a small prediction head to the RSSM. The head would predict either the next physical state or the error between the physics model and the measured next state. An auxiliary loss can then train this head together with the normal DreamerV3 losses:

$$\mathcal{L} = \mathcal{L}_{\text{DreamerV3}} + \lambda_s \|s_{t+1} - \hat{s}_{t+1}\|_2^2. \quad (8)$$

The weight λ_s should start small so the added physical target guides the world model without taking over the original learning objective.

5.6 Reusable skills and one policy for many maps

The MuJoCo simulator and the simple physics model can create short practice tasks before the full maze is used. The policy can learn basic skills such as accelerating, braking, turning, recovering from overshoot, approaching a hole slowly, and handling wall contact. During this pretraining, ball friction, contact parameters, board-response lag, angle calibration, and observation noise should be randomized. The simulator is mainly teaching marble-board skills; the exact response of the real servos and pendulum-coupled frame is learned later from hardware replay.

For transfer to a new maze, the policy must receive local geometry rather than only a map number. TAG already provides a local image and future path points relative to the marble. We can also add the distance and direction to nearby walls and holes. The same policy can then combine reusable motion skills with the geometry of the current map. A small amount of real replay can update the world model for the new board, friction, camera calibration, and pendulum disturbance.

Proposed training path. Use measured board motion and marble trajectories to tune the ball-on-plate part of MuJoCo. Pretrain reusable marble skills across many simulated mazes, then fine-tune on real replay so DreamerV3 learns the actual servo delay, linkage behavior, vibration, and pendulum disturbance.

5.7 Planned comparison

A small ablation study can show whether each addition is useful.

Table 3: Planned physics-guided ablation.

Method	Velocity	Physics input	Residual	Main question
DreamerV3 baseline	No	No	No	How fast does the current system learn?
DreamerV3 + velocity	Yes	No	No	Does measured momentum improve braking and cornering?
DreamerV3 + physics hint	Yes	Yes	No	Does a simple physical prediction reduce real training steps?
Hybrid world model	Yes	Yes	Yes	Does the learned correction improve multi-step prediction?
MuJoCo pretraining	Yes	Sim + physics	Optional	Does simulation reduce the required real steps?
Hybrid + skill pretraining	Yes	Yes	Yes	Can the policy adapt faster to an unseen map?

The main measurements will be physical steps to reach the same success rate, hole rate, completion time, and one-, five-, and ten-step prediction error. We will also measure the simulator-to-hardware trajectory error before transfer. For a new map, we will report the amount of real replay needed before the transferred policy succeeds. This is a future research plan and has not yet been demonstrated on TAG.

6. Conclusion

TAG combines a two-axis labyrinth, camera perception, position-controlled servos, and a top-mounted rotary inverted pendulum. DreamerV3 is the high-level maze controller, while standard controllers remain inside the actuators. The observation includes measured marble velocity so the policy can better understand momentum. Our next step is to tune the MuJoCo marble-board and contact model using measured board angles and real marble trajectories, pretrain across generated mazes, and then fine-tune on the physical robot. We do not expect MuJoCo to reproduce the complete servo-to-board mechanism. That hardware-specific response will be learned from real replay. We will test whether simulation-trained marble skills and a simple physics hint allow one map-conditioned policy to adapt to unseen maze layouts with fewer real training steps.

References

- [1] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, 2025.
- [2] T. Bi and R. D’Andrea, “Sample-efficient learning to solve a real-world labyrinth game using data-augmented model-based reinforcement learning,” arXiv:2312.09906, 2023.
- [3] T. Bi, “CyberRunner software repository,” <https://github.com/thomasbi/cyberrunner>, accessed 2026.
- [4] Control, Optimization, and Online Learning for Autonomy Lab, <https://coolautonomylab.github.io/>, accessed 2026.
- [5] F. Deng, J. Park, and S. Ahn, “Facing off world model backbones: RNNs, Transformers, and S4,” arXiv:2307.02064, 2023.
- [6] M. Ma, T. Ni, C. Gehring, P. D’Oro, and P.-L. Bacon, “Do Transformer world models give better policy gradients?” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [7] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.
- [8] T. D. Nguyen et al., “Multi-maze TAG DreamerV3 and MuJoCo simulation repository,” <https://github.com/dat07032002/Multi-maze>, accessed 2026.